

# HridAI - An AI with Heart

Souvick Das

May 2026

## 1 Introduction

Conversational AI is used in schools, but most systems are not designed for young children’s emotional and safety needs. They often lack proper clinical support, child-friendly communication, and strong safety rules. This work introduces *HridAI* (*an AI with heart*), a structured system for children aged 7–12. It focuses on emotional support, safety, and simple, caring interaction. A key idea in *HridAI* is the *Socratic approach* — it asks questions before giving advice. This helps children understand their feelings and find their own coping ideas instead of just receiving instructions.

Research [1, 2, 3] shows that question-based interaction improves reflection, empathy, and emotional outcomes. It is especially useful for children, whose thinking and self-awareness are still developing. For this reason, *HridAI* guides children to their own solutions first. It only teaches directly when needed, keeping the child active and in control of the conversation.

## Motivation

Primary school children often face emotional challenges such as conflict, anxiety, and family stress, but support is limited. A safe conversational robot could offer early help, support the child, and alert educators when needed.

However, using a large language model (LLM) directly is risky. Without proper controls, it may give advice, make unsafe promises, or miss signs of distress. These behaviours are not acceptable in child settings.

*HridAI* addresses this by using a structured pipeline that enforces strict safety rules, ensuring reliable and safe interaction rather than depending only on the model’s behaviour.

## Major Contributions

- A *layered architecture* separating safety filtering, intent classification, session memory, dialogue policy, response generation, and output validation into independently auditable components. The code repository is publicly available<sup>1</sup>.

---

<sup>1</sup><https://github.com/SutraMind/HridAI>

- *A hard safety gate* based on deterministic, fast regex pattern matching that intercepts critical utterances before any LLM processing occurs, guaranteeing zero model involvement in safeguarding-critical decisions.
- *A finite state machine (FSM) dialogue manager* governs the main phases of the conversation, such as greeting, exploration, holding, and closing. It keeps the dialogue structured by allowing only a small number of predefined states, which reduces complexity and makes the system easier to control and predict.
- *A conservative asymmetric risk scoring system* tracks the level of concern during the conversation. It increases quickly when risky content appears, decreases slowly when the conversation remains safe, and can place the system into a one-way HOLDING state that requires human confirmation to continue.
- *An LLM-as-Judge safety guard* checks every drafted response before it is sent to the child. It ensures that the response is age-appropriate, emotionally safe, and consistent with the safeguarding rules, while correcting minor issues or blocking unsafe replies when necessary.
- *A persistent child profile* stores a limited form of memory across sessions so the system can remember important emotional context over time. This helps the conversation feel continuous without keeping raw conversation logs.

## 2 The Framework

The proposed framework follows a staged pipeline in which safeguarding, interpretation, dialogue planning, and response generation are handled by separate components. Each module has a clearly defined role, which improves transparency, reduces unintended behavior, and makes the system easier to audit.

As shown in Fig. 1, the framework processes each child utterance through a sequence of high-level modules. In this section, each of the components of the framework and their working process have been detailed.

### 2.1 Safety Filtering

The first component is the safety filter. It is applied before the system proceeds with normal dialogue generation. Its role is to detect high-risk phrases or explicit crisis language and trigger immediate escalation when necessary. This is required because some utterances cannot wait for gradual dialogue analysis rather demand a direct safeguarding response. The reason for placing this module early in the pipeline is that safety must not depend on probabilistic generation. If a critical concern is detected, the system should respond through a fixed, verified procedure. This reduces the chance of unsafe delay or inappropriate handling.

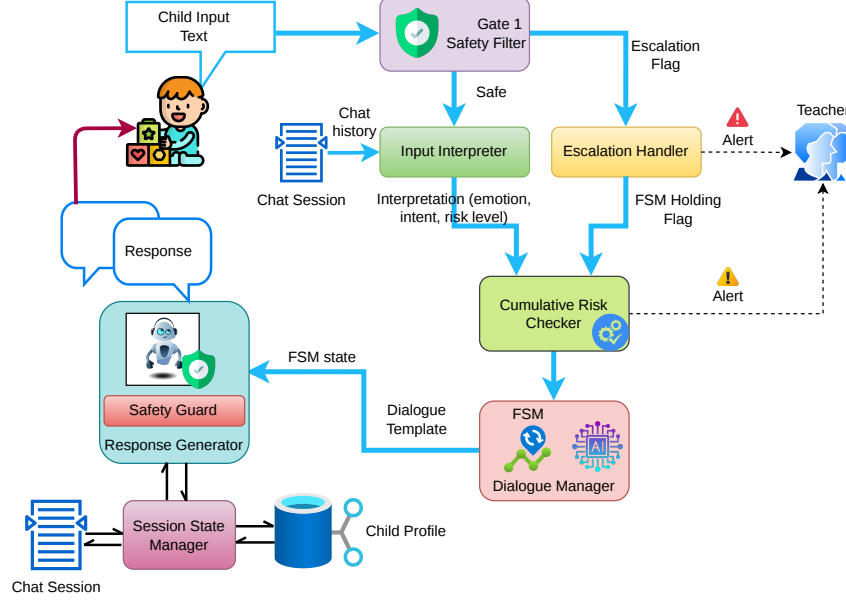


Figure 1: Overview of the HridAI framework.

## 2.2 Escalation Handler

When the safety filter identifies a serious concern, the *Escalation Handler* takes over. This module can notify an educator or place the conversation into a holding state through the FSM, depending on the severity of the situation. The reason for this component is to ensure that unsafe interactions are redirected away from open-ended generation and into a controlled human-in-the-loop response.

## 2.3 Input Interpreter

If no immediate escalation is required, the *Input Interpreter* processes the child’s message to analyze the child’s utterance to identify emotional tone, conversational intent, and any possible concern. This step is required because raw text alone is not sufficient for decision-making. A child may express distress indirectly, use vague language, or switch topics quickly, so the system needs an early interpretation layer to convert unstructured text into usable signals. This step is important because the original utterance is unstructured, while later modules need a more compact representation to make reliable decisions. By converting raw text into interpretable features, the system can respond more consistently to the child’s actual needs.

## 2.4 Cumulative Risk Checker

The Cumulative Risk Checker updates the overall concern level across the session. Its role is to combine the current turn with prior interaction history so that risk is not judged only from a single message. This is required because emotional distress often emerges gradually, and a conservative cumulative view helps the system avoid missing early warning signs. If the risk becomes high enough, the module can raise a soft alert even when the utterance does not trigger immediate escalation. It receives the interpreted emotion, intent, and risk signals from the Input Interpreter, which provide the contextual basis for updating the score. After updating the risk state, it passes the result to the *Dialogue Manager* so that the next conversational move can be selected accordingly.

## 2.5 Dialogue Manager

The Dialogue Manager is the component that decides what the system should do next in the conversation. Its main role is to keep the interaction structured, predictable, and therapeutically safe. Instead of allowing the language model to decide the direction of the dialogue on its own, the framework uses a rule-based controller to choose the current conversation state and the small set of actions that are allowed in that state.

This design is important because an emotional support conversation has a very large number of possible directions. If the model were allowed to freely decide both the policy and the wording at every turn, the system would become difficult to control and difficult to audit. The dialogue manager reduces this state-space problem by restricting the conversation to a few predefined states and a small set of eligible moves inside each state.

In this framework, the rule-based system and the LLM have different responsibilities. The rule-based system decides the phase of the conversation, checks whether a state transition should happen, and selects the next dialogue move using explicit conditions based on session variables. The LLM does not choose the policy; it only receives the selected move and turns it into a natural-language reply that sounds warm and child-appropriate.

More specifically, the Dialogue Manager first reads the current session state, which includes values such as turn count, safety flag, engagement level, response quality, prior moves, and goal completion. It then applies fixed transition rules to determine whether the FSM should remain in the current state or move to the next one. After the state is fixed, it computes the eligible moves for that state and selects one move that best advances the next incomplete conversational goal.

### 2.5.1 How the FSM controls the flow

The FSM contains six states: `GREETING`, `RAPPORT_BUILDING`, `TOPIC_EXPLORATION`, `SKILL_TEACHING`, `CLOSING`, and `HOLDING`. The first five define the normal session flow, while `HOLDING` is a safety override state that can be entered from any point if escalation is required.

The transition rules are deterministic. If the safety flag is `ESCALATE_TO_ADULT`, the manager immediately forces the FSM into `HOLDING`. Otherwise, the normal progression is followed: after the opening exchange, `GREETING` moves to `RAPPORT_BUILDING`; once the child is sufficiently engaged or has introduced a topic, `RAPPORT_BUILDING` moves to `TOPIC_EXPLORATION`; after the exploration phase, the system either moves to `SKILL_TEACHING` if the child cannot generate a coping idea, or to `CLOSING` if the child has already done so.

A minimum turn threshold is also used before leaving `TOPIC_EXPLORATION`. This prevents the session from closing too early and ensures that the conversation has enough space for meaningful exploration before a coping skill is introduced or the interaction is wrapped up.

### 2.5.2 How eligible moves are selected

Once the current FSM state has been decided, the manager restricts the action space to only those moves that are appropriate for that state. For example, in `GREETING` and `RAPPORT_BUILDING`, the system mainly allows simple supportive actions such as validation and reflection. In `HOLDING`, probing and skill teaching are disabled, and only containment-style actions such as reassurance, validation, and holding space are allowed.

In `TOPIC_EXPLORATION`, the manager unlocks the Socratic chain gradually rather than making every move available at once. The sequence begins with `validate`, then `reflect.back`, followed by `socratic_probe`, `socratic_explore`, and `socratic_generate`. The move `teach_skill` is gated and becomes eligible only after the system has already attempted to help the child generate their own coping response and the child’s answer remains vague or absent.

After the eligible set is created, the manager chooses one move by checking the current goal stack. It gives priority to the first incomplete goal and selects the move that best advances that goal. In this way, the rule-based layer controls both the dialogue structure and the therapeutic sequence, while the LLM is used only afterwards to express the selected move in natural language.

### 2.5.3 Role of each state

**GREETING.** This is the entry state of the session. Its role is to open the interaction warmly and briefly, without starting therapeutic work too early. After the initial exchange, the system moves forward because staying in greeting for longer does not add meaningful value.

**RAPPORT\_BUILDING.** This state is used to help the child feel heard and comfortable before deeper exploration begins. The system stays here only long enough to establish trust and confirm that the child is engaged enough to continue.

**TOPIC\_EXPLORATION.** This is the main working state of the conversation. The goal here is to help the child describe feelings, connect them to

situations or patterns, and gradually move toward a child-generated coping idea through the Socratic sequence.

**SKILL\_TEACHING.** This is a fallback state rather than the default path. It is used only when the child could not generate a coping response independently, and the system then offers one concrete strategy in a controlled way.

**CLOSING.** This state ends the session in a warm and stable way. No new exploration is started here; the system only acknowledges what happened and brings the interaction to a safe conclusion.

**HOLDING.** This is the crisis-containment state. When entered, the normal therapeutic flow is suspended, and the system remains supportive while an adult is expected to intervene. The state is one-way within the session, which means the LLM cannot reverse the escalation once the safety system has triggered it.

## 2.6 Response Generator

The Response Generator is responsible for turning the selected dialogue move into a natural-language reply. It does not decide what the system should do next, that decision has already been made by the *Dialogue Manager*. Its role is to express the chosen policy in a warm, simple, and child-appropriate way.

This module is intentionally constrained. The language model receives a prompt that includes the selected move, the current FSM phase, the recent conversation history, the child profile, and relevant session state information. These inputs help the model produce a response that is consistent with the current conversational context rather than generating a generic reply. In this way, the response is shaped by the state of the session instead of being free-form or detached from the ongoing interaction.

The child profile is used to preserve continuity across sessions. It may include stable information such as the child’s name, age, recurring sensitivities, or previous summaries. This allows the robot to respond in a more personal and coherent way without repeating information or ignoring what was already learned about the child. The chat session history is also included so that the response can reflect what was just said and maintain a smooth conversational flow.

The session state manager plays an important role by keeping track of the current phase, the conversation turn, the emotion trajectory, the selected move, and the goal stack. The Response Generator uses this structured state to align the wording of the reply with the current therapeutic purpose. For example, a response in **RAPPORT\_BUILDING** should feel different from one in **TOPIC\_EXPLORATION** or **HOLDING**, even when the child’s utterance is similar.

The generated draft is then passed to the Safety Guard before it reaches the child. This final check is necessary because even a state-guided response can still contain unsafe wording, excessive advice, or a missed distress signal.

The Safety Guard therefore acts as a final control layer: it can allow the draft to pass, silently revise minor issues, or force escalation if the response ignores serious risk.

## 3 Implementation Details

This section describes the technical details of the framework.

### 3.1 Model Usage Strategy

The Gemma 4 26B instruction-tuned model is used in three places — input interpretation, response generation, and LLM-based safety review. It is also used to operate the child profile through a persona-guided prompt. This model was chosen because it is state-of-the-art open-source model that can run on a consumer-graded GPU. Moreover it supports strong instruction following, structured outputs, and long-context processing, which are useful for multi-turn dialogue control and JSON-based intermediate representations.

### 3.2 Gate 1 Safety Filter

The first stage is a deterministic pattern-based safety filter. Its purpose is to detect high-risk expressions before any model inference is performed. This stage is implemented through curated lexical and phrase-level matching for hard-trigger utterances associated with self-harm, immediate danger, abuse, or equivalent crisis language.

The reason for using a deterministic gate here is reliability. For these cases, the system should not depend on probabilistic interpretation. If a hard trigger is matched, the normal flow is bypassed and the escalation route is activated immediately. This guarantees that critical safety decisions are made through explicit rules rather than through model judgement.

### 3.3 Input Interpretation

If the utterance does not activate Gate 1, it is passed to the interpretation stage. Here, Gemma 4 26B is prompted to convert the child’s utterance into a compact structured representation containing *emotion*, *intent*, *risk level*, and *confidence*. The model is therefore used as a semantic classifier rather than as a free-form responder.

The interpretation output is constrained to a small ontology. Emotion is mapped to categories such as **happy**, **sad**, **anxious**, **angry**, **withdrawn**, and **neutral**. Intent is mapped to categories such as **venting**, **sharing**, **deflecting**, **asking**, **topic\_change**, and **distress\_signal**. Risk is similarly reduced to **none**, **low**, or **moderate**. Constraining the output space in this way makes the later rule-based layers easier to design and test.

A prompt-based structured-output format is used so that the model returns machine-readable JSON instead of open prose. This reduces ambiguity and allows the downstream modules to consume the result directly.

#### Interpreter output schema.

```
{
  "emotion": "happy | sad | anxious | angry | withdrawn | neutral |
  unknown",
  "intent": "venting | deflecting | distress_signal | sharing |
  asking | topic_change | unknown",
  "risk_level": "none | low | moderate",
  "confidence": 0.0
}
```

### 3.4 Escalation Handler

When a hard trigger is detected, or when a later safety stage requests escalation, the escalation handler takes control. Its function is to replace the normal conversational path with a fixed safeguarding response. This response can include an educator alert, a forced transition into the **HOLDING** state, and incident logging for traceability.

The escalation path is deliberately non-generative. Once activated, it does not depend on the model to decide whether the event is serious enough or whether the session should continue normally. This is important because escalation is a policy boundary rather than a conversational preference.

### 3.5 Cumulative Risk Checker

The cumulative risk checker combines the interpreted risk from the current utterance with the running session-level concern score. Rather than judging risk from a single turn in isolation, the mechanism accumulates concern gradually over time. This is useful because emotional distress often appears as a pattern across several turns rather than as a single explicit statement.

The score update is asymmetric: concern increases faster than it decreases. Low-risk turns contribute a small increment, while moderate-risk turns contribute a larger increment. Decay occurs only after a sustained safe streak, and the score can retain a floor based on the session peak after serious concern. This conservative design prevents the system from returning to a fully normal policy too quickly after a concerning interaction.

If the cumulative score crosses a predefined soft-alert threshold, a teacher-facing warning is generated even if no hard trigger was matched. Otherwise, the updated risk state is passed forward to the dialogue manager.

### 3.6 Dialogue Manager and FSM Control

The dialogue manager is implemented as a rule-governed finite state controller. Its role is to determine the current conversational phase and the set of allowed dialogue moves for that phase. This is done using the interpreted turn signal together with session variables such as turn count, engagement level, response quality, prior moves, goal completion, and safety flag.

The use of an FSM solves two implementation problems. First, it limits the dialogue state space to a small number of interpretable macro-states. Second, it makes the therapeutic progression explicit, so that the system cannot skip directly from a greeting to advice-giving without first passing through rapport and exploration.

The state set used in the framework is:

GREET  $\rightarrow$  RAPPORT\_BUILD  $\rightarrow$  TOPIC\_EXPLORE  $\rightarrow$  SKILL\_TEACH  $\rightarrow$  CLOSE

with HOLDING reachable from any state as a safety override.

| Current state     | Next state        | Trigger condition                                                                                                                           |
|-------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Any state         | HOLDING           | Safety flag indicates escalation to adult                                                                                                   |
| GREETING          | RAPPORT_BUILDING  | At least one turn completed                                                                                                                 |
| RAPPORT_BUILDING  | TOPIC_EXPLORATION | At least two turns completed and child is engaged, or a topic has already been raised                                                       |
| TOPIC_EXPLORATION | SKILL_TEACHING    | Socratic generation already attempted, child response remains vague or absent, solution goal incomplete, and minimum turn threshold reached |
| TOPIC_EXPLORATION | CLOSING           | Child-generated solution goal completed and minimum turn threshold reached                                                                  |
| SKILL_TEACHING    | CLOSING           | Skill-teaching move already delivered                                                                                                       |
| CLOSING           | CLOSING           | Terminal state                                                                                                                              |

Table 1: Finite state transition logic used by the dialogue manager.

**FSM transition rules.** Once the phase is fixed, the dialogue manager computes the eligible moves for that state. For example, GREETING and RAPPORT\_BUILDING permit supportive moves such as validation and reflection, while TOPIC\_EXPLORATION unlocks the Socratic sequence gradually. The move `teach_skill` is gated and becomes available only when the child has not been able to generate a coping idea independently. This keeps the system from giving advice too early.

### 3.7 Response Generation

The response generator uses Gemma 4 26B to transform the selected dialogue move into a natural-language reply. Importantly, the model does not decide the dialogue policy. At this stage, the state and move have already been fixed by the rule-based dialogue manager. The model is used only to realise that decision in a warm, child-appropriate form.

The prompt for this stage includes four elements: the selected move directive, the current FSM phase, the recent conversation history, and profile-based child context. The move directive acts as a local behavioural policy. For example, a validation move instructs the model to acknowledge the child’s feeling without giving advice, whereas a Socratic move instructs it to ask one specific kind of question. This makes the generated text state-oriented rather than generic.

The recent session history ensures local coherence, while the child profile provides longitudinal continuity. Together, these inputs help the model produce responses that are consistent with both the current turn and the broader interaction history.

### **Prompt structure for response generation.**

```

Persona block
+ current FSM phase
+ move-specific directive
+ child profile summary
+ recent conversation turns
+ current child utterance
=> draft response

```

## **3.8 Safety Guard as LLM-as-Judge**

After a draft response is generated, it is passed to a second Gemma 4 26B prompt that acts as a safety reviewer. This module functions as an LLM-as-Judge. Instead of creating a new reply from scratch, it evaluates the generated draft against a fixed checklist and returns a structured verdict.

The review criteria include whether the response uses diagnostic language, promises secrecy, gives advice in list form, asks too many questions, uses vocabulary inappropriate for a child, or ignores a distress signal that should have led to escalation. This design adds a final verification layer after generation, which is important because even a constrained generator can occasionally produce a problematic response.

The guard produces one of three outcomes: pass the draft unchanged, revise the draft silently for non-critical issues, or request escalation if the child’s utterance contained serious danger that the draft failed to address. In this way, the final output is checked not only at the policy level but also at the wording level.

### **Safety judge output schema.**

```

{
  "pass": true,
  "violations": [],
  "revised_response": null,
  "safety_flag": "NONE | TOPIC_SENSITIVE | ESCALATE_TO_ADULT |

```

```

    CONTENT_VIOLATION"
}

```

### 3.9 Child Profile and Session Representation

Persistent child context and temporary session context are both maintained in JSON format. JSON is used because it is lightweight, human-readable, easy to inspect, and directly compatible with prompt assembly and structured model outputs.

The child profile stores longer-term information that should persist across sessions, such as recurring sensitivities, recurring emotional themes, and short summaries of prior sessions. This profile is updated using Gemma 4 26B with a persona-guided summarisation prompt so that the memory remains compact and behaviourally relevant rather than becoming a raw transcript archive.

#### Child profile schema.

```

{
  "child_id": "string",
  "name": "string",
  "age": 0,
  "previous_topics_discussed": ["string"],
  "recurring_emotions": ["string"],
  "known_sensitivities": ["string"],
  "session_history": ["string"]
}

```

The session representation stores the evolving state of the current interaction. It contains the current FSM phase, the running risk score, goal progress, prior moves, recent dialogue history, engagement indicators, and safety flags. Unlike the child profile, this structure is short-lived and is reset when a new session begins.

#### Session state schema.

```

{
  "child_id": "string",
  "session_id": "string",
  "turn_count": 0,
  "fsm_phase": "GREETING | RAPPORT_BUILDING | TOPIC_EXPLORATION | SKILL_TEACHING | CLOSING | HOLDING",
  "emotion_trajectory": ["string"],
  "topics_raised": ["string"],
  "moves_used": ["string"],
  "engagement_level": "LOW | MODERATE | HIGH",
  "deflection_count": 0,
  "child_gave_content": false,
}

```

```

"pattern_named": false,
"child_response_quality": "CLEAR | VAGUE | NONE",
"goal_stack": [
  {"name": "string", "done": false}
],
"safety_flag": "NONE | TOPIC_SENSITIVE | ESCALATE_TO_ADULT |
CONTENT_VIOLATION",
"conversation_history": [
  {"role": "child | robot", "text": "string"}
],
"loop_counter": {
  "move_name": 0
},
"cumulative_risk_score": 0,
"last_intent": "string"
}

```

### 3.10 Profile Updating

At the end of a session, the accumulated dialogue trace is summarised into a compact update for the child profile. This summarisation is again performed through a constrained prompt using the same model, with a persona-based instruction that preserves only stable, useful context. The aim is not to store everything the child said, but to retain what may matter for future supportive interactions.

This approach keeps memory bounded and interpretable. It also reduces privacy risk compared with storing raw logs while still allowing the system to maintain continuity across sessions.

## 4 Limitations

The current implementation has several limitations that affect its reliability and generalisability.

- **Zero-shot classification.** Emotion, intent, and risk level are interpreted through zero-shot prompting without any task-specific fine-tuning, which may produce inconsistent results for indirect or ambiguous child utterances.
- **Regex-based safety filter.** Gate 1 relies on pattern matching and may fail to detect crisis-level content expressed through indirect language, metaphor, or phrasing not covered by the predefined patterns.
- **General-purpose safety judge.** The Safety Guard uses a general-purpose instruction-tuned model rather than a fine-tuned specialized one.

- **Fixed dialogue structure.** The FSM enforces a rigid session progression that may not adapt well to children whose emotional needs fall outside the expected interaction arc.

## 5 Possible Improvements

To strengthen both safety and effectiveness, the framework can be improved along the following practical directions:

- **Hybrid safeguarding filter (rules + classifier).** The regex-based safety filter can be replaced with a hybrid module. The module may combine regex for high-precision hard rules, and add a lightweight classifier to detect indirect, metaphorical, or context-dependent risk expressions (e.g., “I don’t want to be here” or “everything feels pointless”).
- **Stronger emotion/intent/risk recognition.** Another improvement includes the enhancement of the *Input Interpreter* by fine-tuning or adapting models on age-relevant data for recognizing the emotion, intent and risk level. This can reduce confusion between similar intents (e.g., “seeking reassurance” vs. “seeking advice”), and improve handling of short, vague child utterances.
- **Domain-specific safety guardrail.** Curation of a child-safeguarding dataset that includes unsafe requests, boundary testing, disclosure of harm, bullying, and secrecy cues could be another direction. Following this, fine-tuning of LLM for safety classification may enhance the overall quality of the framework. Alternatively, the existing LLM guards such as Nvidia Nemo [5], LLama-guard [4] can be integrated with the framework.
- **Expert-defined FSM policies and dialogue style.** Another crucial enhancement can be achieved by refining the FSM transition rules with domain experts (e.g., school counselors, child psychologists). This includes: (i) adding clearer entry/exit criteria for each state and (ii) specifying what the robot *should never do* in each state. Expert input can also tune the tone (warmth, pacing, simplicity) and the balance between questions and reflection.
- **Child profile memory via LLM-Wiki + knowledge graph.** The framework can be enhanced by using the LLM-Wiki<sup>2</sup> pattern to maintain a persistent set of interlinked Markdown pages (e.g., “Child”, “People”, “Topics”) that are incrementally updated across sessions, rather than re-deriving context each time. In parallel, a lightweight knowledge graph can be integrated alongside the LLM-Wiki, where *entities* are represented as nodes (e.g., child, teacher, emotion, event, objects, characters) and *relationships* are represented as labeled edges (e.g., `triggered_by`, `prefers`,

---

<sup>2</sup><https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

`supports`, `conflict_with`). This hybrid design keeps long-term memory consistent, auditable, and easy to query, while allowing the LLM to use the graph structure to preserve and reason over nuanced context in a structured and reliable way.

Along with this memory framework, we may implement the dream mode for organizing the memory. During idle time, the *dream cycle* will consolidate memory by repairing broken links, merging duplicates, resolving contradictions, and pruning stale entries.

## 6 Future Work

Future work will focus on following directions:

- **Multi-turn child-safe evaluation benchmark.** Building a benchmark that tests multi-turn reasoning in child social-emotional settings (e.g., conflict, worry, exclusion, shame). Evaluate prompting and reasoning styles such as zero-shot, few-shot, chain-of-thought style prompting, tree-based reasoning, self-reflection, and Socratic chains.
- **End-to-end framework iteration using benchmark results.** Use benchmark outcomes to iteratively improve each module (safety filter, interpreter, risk aggregation, FSM policy, response templates).
- **Pilot studies and human-in-the-loop review.** Run controlled studies with educators and child-development experts using simulated or ethically approved scenarios. Include transcript review to validate safeguarding decisions, and refine escalation thresholds and “soft alert” policies to fit school practice.

## References

- [1] Carlos Carona, Charlotte Handford, and Ana Fonseca. Socratic questioning put into clinical practice. *BJPsych Advances*, 27(6):424–426, 2021.
- [2] Philip Held, Sarah A Pridgen, Yaozhong Chen, Zuhaib Akhtar, Darpan Amin, and Sean Pohorence. A novel cognitive behavioral therapy-based generative ai tool (socrates 2.0) to facilitate socratic dialogue: Protocol for a mixed methods feasibility study. *JMIR Research Protocols*, 13(1), 2024.
- [3] Philip Held, Sarah A Pridgen, Daniel R Szoke, Yaozhong Chen, Zuhaib Akhtar, and Darpan Amin. Ai-facilitated cognitive reappraisal via socrates 2.0: Mixed methods feasibility study. *JMIR Mental Health*, 12, 2025.
- [4] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.

- [5] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations*, pages 431–445, 2023.